Практична робота

Тема: ***Оброблення виняткових ситуацій.***

Виконав: **Устич Максим**

Група : **alk-43**

Київ 2026р.

Виконання завдання

У ході виконання лабораторної роботи було створено програму мовою C++, у якій реалізовано оброблення виняткових ситуацій за допомогою операторів **`try`**, **`catch`** і **`throw`**.

Було розроблено клас **`Vect`**, який використовується для роботи з динамічним масивом цілих чисел.

У програмі реалізовано:

- перевірку допустимого розміру масиву в конструкторі;
- оброблення винятків у конструкторі;
- оброблення винятків у деструкторі;
- перевірку виходу індексу за межі масиву;
- перехоплення винятків у функції **`main()`**.

При створенні об'єкта класу перевіряється значення розміру масиву. Якщо розмір менший або дорівнює нулю чи перевищує 256, генерується виняткова ситуація за допомогою оператора **`throw`**.

Для демонстрації роботи винятків у програмі навмисно створюється об'єкт із некоректним розміром масиву (300), що викликає помилку, яка перехоплюється в блоці **`catch`**.

Лістинг програми

```
#include <iostream>
#include <windows.h>
#include <clocale>

using namespace std;

class Vect
{
public:
    Vect(int n) : size(n), p(nullptr)
    {
```

```
        try
        {
            if (size <= 0 || size > 256)
                throw "Невірний розмір масиву!";

            p = new int[size];

            for (int i = 0; i < size; ++i)
                p[i] = 0;
        }
        catch (...)
        {
            cout << "Помилка у конструкторі Vect" << endl;
            throw;
        }
    }

    ~Vect()
    {
        try
        {
            delete[] p;
        }
        catch (...)
        {
            cout << "Помилка у деструкторі Vect" << endl;
        }
    }

    int& operator[](int i)
    {
        if (i < 0 || i >= size)
            throw "Індекс поза межами масиву!";
        return p[i];
    }
```

```
    void Print()
    {
        for (int i = 0; i < size; ++i)
            cout << p[i] << " ";
        cout << endl;
    }

private:
    int* p;
    int size;
};

int main()
{
    SetConsoleOutputCP(65001);
    setlocale(LC_ALL, "");
```

```
    try
    {
        Vect a(3);
        a[0] = 0;
        a[1] = 1;
        a[2] = 2;
        a.Print();

        Vect a1(300); // спеціально помилка
        a1[10] = 5;
        a1.Print();
    }
    catch (const char* s)
    {
        cout << "Помилка: " << s << endl;
    }
    catch (...)
    {
        cout << "Невідома помилка!" << endl;
    }

    return 0;
}
```

Результати виконання програми

Lab_Exception_Vect Vect

```
#include <iostream>
using namespace std;

class Vect
{
public:
    Vect(int n) : size(n), p(nullptr)
    {
        try
        {
            if (size <= 0 || size > 256)
                throw "Невірний розмір масиву!";

            p = new int[size];

            for (int i = 0; i < size; ++i)
                p[i] = 0;
        }
        catch (...)
        {
            cout << "Помилка у конструкторі Vect" << endl;
            throw;
        }
    }
```

```cpp
    ~Vect()
    {
        try
        {
            delete[] p;
        }
        catch (...)
        {
            cout << "Помилка у деструкторі Vect" << endl;
        }
    }

    int& operator[](int i)
    {
        if (i < 0 || i >= size)
            throw "Індекс поза межами масиву!";
        return p[i];
    }

    void Print()
    {
        for (int i = 0; i < size; ++i)
            cout << p[i] << " ";
        cout << endl;
    }
private:
    int* p;
    int size;
};

int main()
{
    try
    {
        Vect a(3);
        a[0] = 0;
        a[1] = 1;
        a[2] = 2;
        a.Print();

        Vect a1(200);
        a1[10] = 5;
        a1.Print();
    }
    catch (const char* s)
    {
        cout << "Помилка: " << s << endl;
    }
    catch (...)
    {
        cout << "Невідома помилка!" << endl;
    }

    return 0;
}
```

Рис. 1 – Код програми з обробленням виняткових ситуацій

Виконання програми без помилки

При використанні допустимого розміру масиву програма працює коректно та виводить значення елементів масиву.

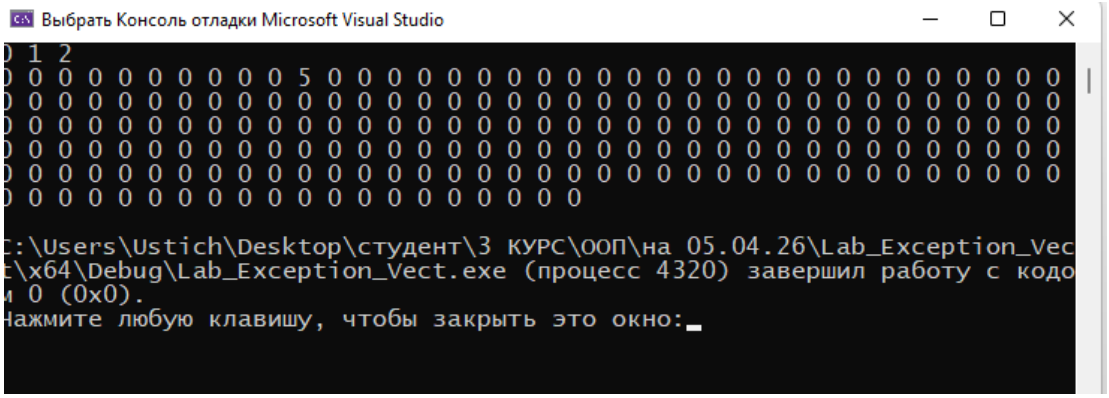

Рис. 2 – Результат виконання програми без виникнення виняткової ситуації

Виконання програми з помилкою

При використанні некоректного розміру масиву (300) генерується виняткова ситуація, яка перехоплюється в блоці catch. У результаті виводиться повідомлення про помилку.

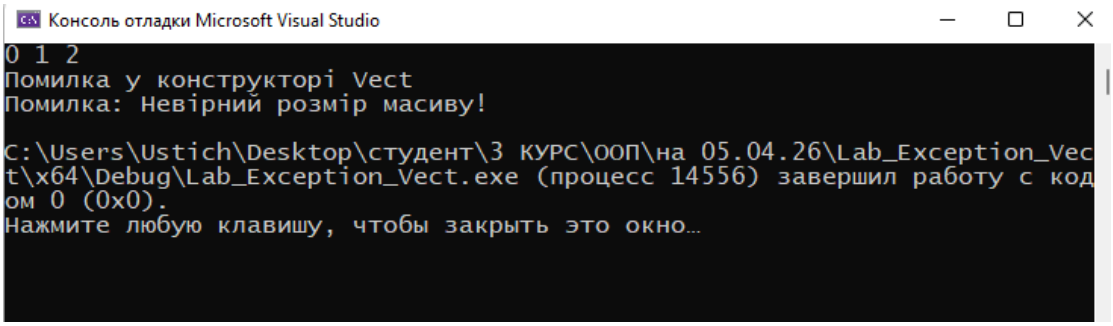

Рис. 3 – Результат виконання програми з обробленням виняткової ситуації

Висновок

У ході виконання роботи було реалізовано оброблення виняткових ситуацій у мові програмування C++.

Було використано оператори **try**, **catch** і **throw**, що дозволило контролювати помилки під час виконання програми та забезпечити її стабільну роботу.